Stablization of module customizations

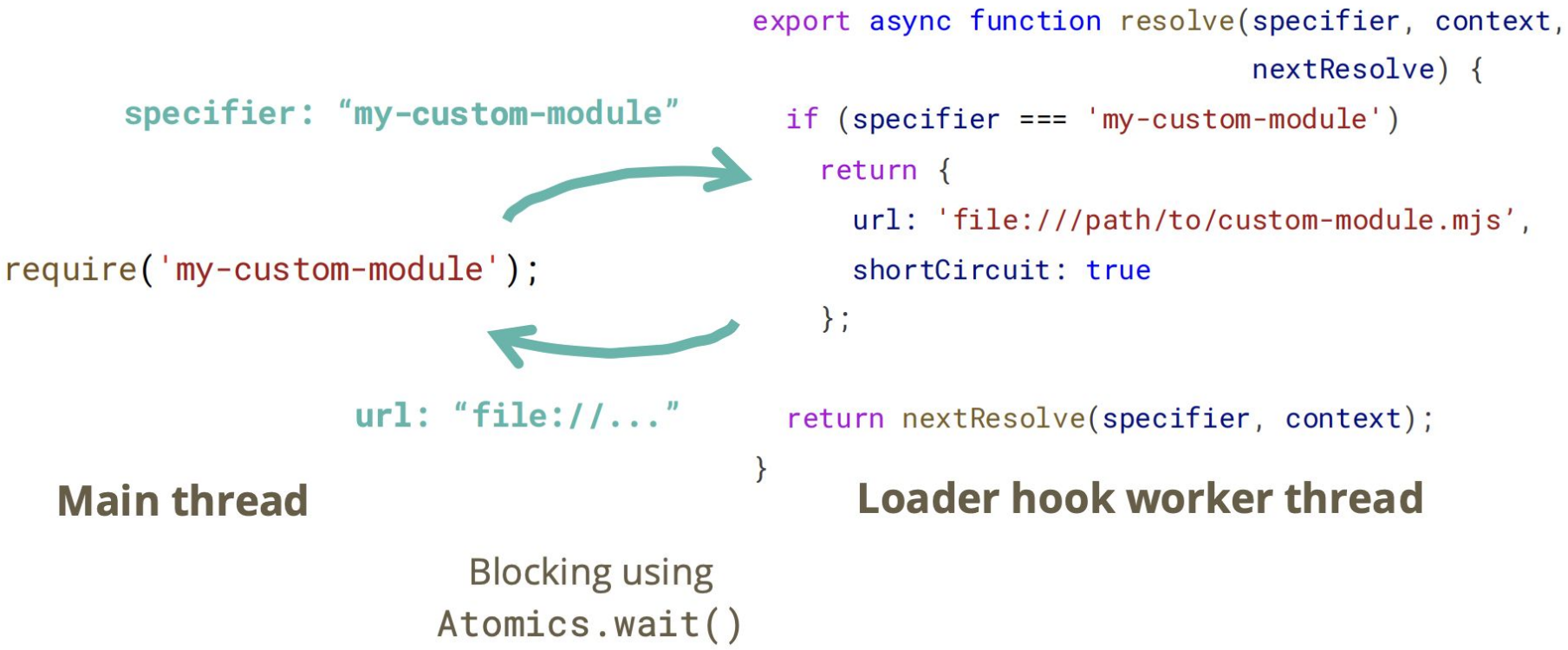
Joyee Cheung

module.register()

- If you don’t know who they are: basically the same module loading hooks, one async + off-thread, one sync + in-thread

```
// async-hook-module.mjs
export async function resolve(specifier, context, nextResolve) {
  // Take an `import` or `require` specifier and resolve it to a URL.
}
export async function load(url, context, nextLoad) {
  // Take a resolved URL and return the source code to be evaluated.
}
// async-hooks.mjs
import { register } from 'module';
register('./async-hook-module.mjs', import.meta.url);
```

module.register()



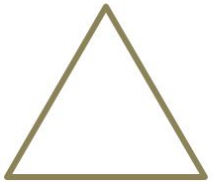
module.register -> module.registerHooks migration

- If you don’t know who they are: basically same module loading hooks, one async + off-thread, one sync + in-thread

```
// sync-hooks.mjs
import { registerHooks } from 'module';
function resolve(specifier, context, nextResolve) {
  // Take an `import` or `require` specifier and resolve it to a URL.
}
function load(url, context, nextLoad) {
  // Take a resolved URL and return the source code to be evaluated.
}
registerHooks({resolve, load});
```

module.register & module.registerHooks

~~Works for all modules~~



Async **In thread**

`--experimental-loader &`
`module.register()` before v20

Works for all modules*

Async ~~In thread~~

`--experimental-loader &`
`module.register()` after v20

Works for all modules

~~Async~~ **In thread**

New API:
`module.registerHooks()`

module.register & module.registerHooks

- Current status: two loading paths internally in the module loader
  - The one used by module.register() re-invents require() with a lot of quirks compared to the normal require()
  - The rest reuses CJS loader as much as possible when it loads CJS and try to do things synchronous if there’s no top-level await

Quirks of module.register()

- Asynchronous hooks run on a separate thread, so the hook functions cannot directly mutate the global state of the modules being customized. It's typical to use message channels and atomics to pass data between the two or to affect control flows. See Communication with asynchronous module customization hooks .
- Asynchronous hooks do not affect all `require()` calls in the module graph.
  - Custom `require` functions created using `module.createRequire()` are not affected.
  - If the asynchronous `load` hook does not override the `source` for CommonJS modules that go through it, the child modules loaded by those CommonJS modules via built-in `require()` would not be affected by the asynchronous hooks either.
- There are several caveats that the asynchronous hooks need to handle when customizing CommonJS modules. See asynchronous `resolve` hook and asynchronous `load` hook for details.
- When `require()` calls inside CommonJS modules are customized by asynchronous hooks, Node.js may need to load the source code of the CommonJS module multiple times to maintain compatibility with existing CommonJS monkey-patching. If the module code changes between loads, this may lead to unexpected behaviors.

Issues with maintaining both

- This adds maintenance burden: two sets of everything, done slightly differently
- Bugs/quirks in module.register() are harder to solve / cannot be solved due to the the threading design
- Technically you can re-implement most of module.register() with module.registerHooks() + worker threads + atomics in the user land, but not the other way around

Deprecation of module.register()

- Goal: make it simple again, and close the unresolvable bugs
- doc deprecated for <=25: https://github.com/nodejs/node/pull/62395
- runtime warning for >=26: https://github.com/nodejs/node/pull/62401

What’s the migration path?

- Ponyfill for drop-in replacement: https://github.com/joyeecheung/module-register-ponyfill
- Most of module.register() could’ve been implemented in the userland. Just moves most of the worker coordination code in core outside to be its own package

```
// Before:
import { register } from 'node:module';
register('./hooks.mjs', import.meta.url);

// After:
import { register } from 'module-register-ponyfill';
register('./hooks.mjs', import.meta.url);
```

Ponyfill?

Also works as polyfill

- `node --import module-register-ponyfill/polyfill app.js`
- Patches module.register(), mostly for those who cannot control the hooks e.g. in a package

Ponyfill?

- Earlier register() calls do not affect the loading of later hook modules in the worker
  - It otherwise requires in-thread async customization on the loader worker thread which was what makes everything quirky in the first place
  - That part is special cased in core, and would leak too many details if exposed

```
module.register('./load-from-zip.js', import.meta.url);
module.register('./hooks.zip', import.meta.url);
```

Status

- Ponyfill
  - Pass most of the core tests ported
  - Pass tests of import-in-the-middle (~1 month ago)
  - Require a few features in core to reach feature parity (e.g. worker argument filtering)
  - Plan to move it into the organization and publish it soon when those get fixed
- Stability index
  - module.register() now deprecated (0)
  - module.registerHooks() now release candidate (1.2)
  - Expect to collect feedback, fix any remaining bugs during and promote module.registerHooks() to stable by 26 LTS

Status

- Some remaining feature requests for module.registerHooks()
  - evaluate/link hooks
  - Defer to the next section vm module loader customization
- Thoughts about the migration plan?
- What can we do to help the ecosystem migrate & stop monkey patching?
  - E.g. sending PRs https://github.com/danez/pirates/pull/130
  - Getting help from e18e
  - Other ideas?

Stablization of vm modules

- --experimental-vm-modules, vm.SourceTextModule, vm.SyntheticModule, importModuleDynamically options in e.g. vm.Script
- If you are not familiar with them, SourceTextModule are JS modules and what people called ESM usually, SyntheticModules are facades in front of CJS, Node.js builtins, JSON modules etc.
- They map to classes in the ECMA262 specification
- The vm classes wrap around v8::Module implementations
- Used by e.g. testing frameworks like Jest and Vitest to run modules in a separate context (for test isolation)

Stablization of vm modules

- Experimental since v9 in 2017
- Main reason why Jest still doesn’t support ESM well https://github.com/jestjs/jest/issues/9430 - in turn still blocks ESM adoption in the ecosystem to some degree
- Vitest worked around it by providing child-process/worker-thread-based isolation, but they also have a mode that uses context-based isolation and wants to make use of it better when it statblizes

Stablization of vm modules

- Stablization issue is 6-year old: https://github.com/nodejs/node/issues/37648
- Accumlated several issues over the years
  - Memory management: leak and use-after-frees
  - Linking was driven by core, invoked a user callback, forced asynchronicity (i.e. couldn't be used to re-implement require(esm)) and incurs many C++ -> JS callbacks
  - Still requires a lot of plumbing if you only need partial customization

Trying to fix SourceTextModule recently

- Fixing memory management: works correctly, but GC-inefficient
- Dynamic import customization
  - Now can be configured on the vm context, not just per-module for better memory management

Trying to fix SourceTextModule recently

```
const foo = new vm.SourceTextModule(`import 'a'; import 'b'`, { context });
// Invoked with specifier 'a' and 'b' respectively, and referencingModule `foo`.
async function linker(specifier, referencingModule) {
  return new vm.SourceTextModule(`export default ${specifier};`, { context });
}
await foo.link(linker);
await foo.evaluate();  // returns a Promise that fulfills with the namespace object of `foo`
```

Trying to fix SourceTextModule recently

```
const foo = new vm.SourceTextModule(`import 'a'; import 'b'`, { context });
const modules = foo.moduleRequests.map(({ specifier }) => {
  return new vm.SourceTextModule(`export default ${specifier};`, { context });
});
// Does early checks, doesn't instantiate, you can do DFS/BFS as you wish
foo.linkRequests(modules);
foo.instantiate();
// Now returns an immediately resolved promise if it has no TLA
foo.evaluate();
// Also it now has:
foo.hasTopLevelAwait(); // false
foo.hasAsyncGraph();  // false
// When those are false, you can check foo.error for evaluation errors synchronously.
```

Problem

- Most parts of the API has been re-invented and squeezed into the class in a non-breaking way
- It’s confusing to have two orthogonal ways to do everything in the same class
- We ended up recycling names of previously removed methods e.g. instantiate() and broke Webpack’s test harness

Proposal

- Current constructors are directly exposed in vm (via --experimentla-vm-modules)
- Expose a new SourceTextModule in vm/module, without the old parts
- SynthethicModule mostly stays the same and also exposed in vm/module
- Both no longer receives importModuleDynamically/initializeImportMeta
  - This should be per-context, not per-module
- Try to address all the remaining unsolved issues in the new API: SourceTextModuleLoader as the main customization entrypoint, module constructors are primitives

Look at the new API draft

https://github.com/nodejs/node/issues/62720